

Campus Management System

Project Execution Plan & Responsibility Matrix

INSTITUTION	AUTHORS / TEAM	TARGET LANGUAGE
FAST NUCES, Lahore	Anas & Huzaifa	Java (JDK 17+)

1. EXECUTIVE SUMMARY & ARCHITECTURE OVERVIEW

This project involves designing and implementing a robust, object-oriented **Campus Management System** tailored to support core academic workflows at FAST National University. Built strictly upon standard Object-Oriented Programming (OOP) principles in Java, the architecture models five core actor roles: *Academic Office Admin*, *Permanent Instructor*, *Visiting Instructor*, *Teaching Assistant (TA)*, and *Normal Student*.

Core System Requirements & Persistence Mandate

All application state must be stored persistently in plain text files inside the `data/` folder. Dynamic in-memory operations are backed by structured file I/O operations. Furthermore, the system maintains a real-time event logging engine writing operational metrics, exceptions, and user actions into `logs/app.log`.

2. TEAM WORK DIVISION & RESPONSIBILITY MATRIX

DOMAIN / SUBSYSTEM	PRIMARY OWNER	ASSIGNED DELIVERABLES & TECHNICAL SCOPE
Academic Core Infrastructure	ANAS	Design and implement Course, Section, Schedule, and Enrollment models. Build section capacity rules, prerequisite verification, and schedule clash detection (<code>hasClash</code> logic).
Academic Requests Subsystem	ANAS	Develop the abstract Request class alongside <code>CourseClashRequest</code> and <code>GenericRequest</code> models. Implement Admin approval and rejection workflow engines.
Persistence & Logging Engine	ANAS	Construct custom text-file parsing and serialization engine reading/writing to <code>data/*.txt</code> files. Build application-wide Logger utility appending system events to <code>logs/app.log</code> .
User & Role Hierarchy	HUZAIFA	Implement base Person class and specialized subclasses: Student, NormalStudent, TeachingAssistant, Instructor, VisitingInstructor, PermanentInstructor, and AcademicOfficeAdmin.
Assessment & Grading Subsystem	HUZAIFA	Build Assessment abstract class, Assignment, Submission, and Feedback classes. Implement TA grading workflows, deadline checks, and late submission markers.
FYP Management Subsystem	HUZAIFA	Implement FYPGroup, FYPMeeting, and FYPEvaluation models. Enable Permanent Instructor supervision features including meeting scheduling and idea evaluations.
Exceptions & Comparators	HUZAIFA	Construct custom exception hierarchy rooted in <code>CampusException</code> (e.g. <code>CourseFullException</code> , <code>SubmissionDeadlineException</code>).

3. 5-DAY IMPLEMENTATION ROADMAP

DAY	FOCUS AREA	ANAS (CORE, REQUESTS & FILE I/O)	HUZAIFA (USERS, EVALUATION & FYP)
Day 1	Setup & Foundations	Initialize Git repository, folder structure (data/, logs/). Implement Day, EnrollmentStatus, and AttendanceStatus enums.	Create RequestStatus, RequestCategory, and SubmissionStatus enums. Implement Person base class and Student model.
Day 2	Domain Object Models	Build Course, Section, Schedule, and Enrollment domain classes. Implement Schedule.hasClash() algorithm.	Build Instructor hierarchy (PermanentInstructor, VisitingInstructor), TeachingAssistant, and NormalStudent models.
Day 3	Subsystems & Workflows	Implement abstract Request, CourseClashRequest, and GenericRequest. Build Admin request approval/rejection methods.	Implement Assessment, Assignment, Submission, and Feedback. Build FYPGroup, FYPMeeting, and FYPEvaluation modules.
Day 4	I/O, Logging & Exceptions	Develop text file persistence handler for data/ folder. Implement centralized Logger writing to logs/app.log.	Implement CampusException hierarchy (all domain exceptions). Implement custom Comparator classes for sorting.
Day 5	CLI Driver & Testing	Implement Interactive CLI menus for AcademicOfficeAdmin and Student roles. Lead end-to-end integration testing.	Implement Interactive CLI menus for Instructor and TeachingAssistant roles. Conduct code cleanup and add Javadoc comments.

4. ARCHITECTURE & TECHNICAL GUIDELINES

- **Git Workflow & Collaboration:** Development will proceed on separate branches (feature/anas-core and feature/huzaifa-users). Merges into main require mutual code review. Instructor GitHub accounts (thomas-catt, m-ahmad-butt) must be added as private repository collaborators.
- **File Handling Rules:** Application data must be read from and saved to plain text files located strictly inside the data/ folder (e.g., data/courses.txt, data/users.txt). Hardcoding operational data in memory is forbidden.
- **Logging Protocol:** Every state-changing operation (course creation, registration, grade submission, request processing) and runtime exception must be formatted and appended to logs/app.log with ISO timestamps.
- **Exception Handling Strategy:** Business rules must enforce custom exception throwing (e.g., throw CourseFullException when registering for a section with 0 capacity; throw UnauthorizedActionException if a Visiting Instructor attempts to assign a TA).
- **Academic Integrity Mandate ("Vibe Coding" Policy):** Generating unverified AI code without full comprehension is strictly prohibited. Both Anas and Huzaifa must understand every line of code submitted and be prepared to explain all underlying OOP design choices.

Final Integration Checklist

- ✓ All 5 primary actor workflows verified via CLI driver.